

Clinical Trial Matching Copilot

Designing agent systems that know when they are
not sure.

live demo · langgraph + groq · richard abishai

↗ live demo

**LangGraph + Groq, built
to admit doubt.**

Multi-agent AI

architecture, live

/_agenda_

Agents That Know When They Are Not Sure

CAN ONE AI SAFELY DECIDE IF A PATIENT FITS A CLINICAL TRIAL?

THE INPUT

patient notes; messy free text
trial rules; long and brittle
missing data; extremely common

THE FAILURE

one model fills gaps
with guesses
then sounds confident
and certain
we cannot see where judgment
slipped

One model fills the gaps.
Then it sounds certain anyway.

A few terms, in plain words

Agent

an LLM given one job and rules

State

a shared whiteboard agents read and write

Orchestration

deciding which agent runs next

JSON contract

a strict shape for each answer

Temperature

creativity dial; zero is most predictable

Inference

the model thinking and producing output

ACT 1

THREE AGENTS

Split the one opaque step into
three honest, inspectable ones.

The Researcher –

extracts facts from the text.
It never decides eligibility.

The Evaluator –

judges using only the evidence,
and is allowed to say: not sure.

The Orchestrator –

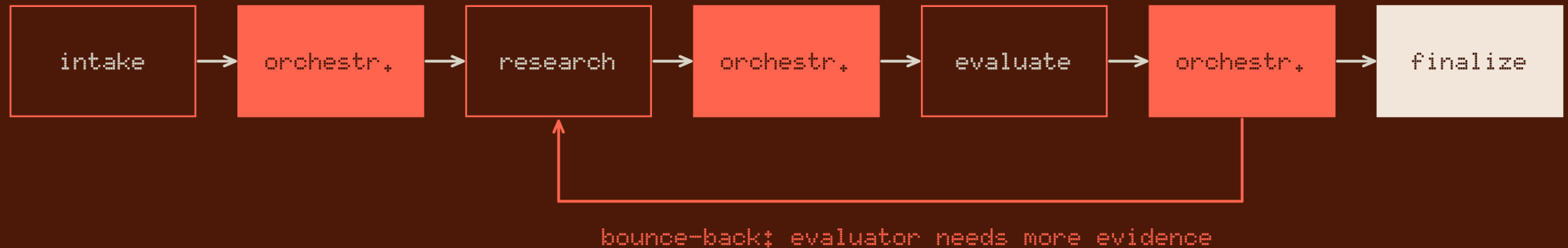
routes work between the agents,
handles the retries and the finish.

ACT 2

THE WIRING

How the agents are wired

shared state flows through each node



state plus nodes plus edges
orchestrator sits between every step

retry logic is explicit, not hidden
every decision is visible, debuggable

Code 1/4 · Shared state

the whiteboard every agent reads and writes

```
class TrialState(TypedDict):  
    patient_case: str  
    trial_criteria: str  
    evidence_pack: dict  
    evidence_gaps: list  
    evaluator_report: dict  
    final_recommendation: str  
    next_step: str  
    retry_count: int  
    bounce_reason: str
```

what this does

one typed object, shared by all
each node returns a small update
state in, partial state out
bounce_reason carries the
retry question

Code 2/4 · Two specialists

researcher extracts; evaluator judges and may refuse

```
# researcher: extract, never decide
return {"evidence_pack": parsed,
        "evidence_gaps": gaps}

# evaluator: judge from evidence
verdict in [
    "eligible",
    "unlikely_eligible",
    "unclear",
    "needs_more_evidence"]
```

what this does

researcher only structures the facts
it must list gaps, not hide them
evaluator sees evidence, never
raw notes
it can return needs_more_evidence
that is permission to be uncertain

Code 3/4 · The orchestrator

the brain: it routes, and it handles the bounce-back

```
# no evidence yet -> research
if not evidence_pack:
    next = "research"

# evaluator refused -> retry
if verdict == "needs_more_evidence"
    and retry_count < 1:
    next = "research"
    bounce_reason = question
    retry_count += 1
```

what this does

reads the state, picks the next step
evaluator says not enough? work goes
back to research
the evaluator's question rides along
the retry counter prevents
endless loops

Code 4/4 · Build the graph

add nodes, wire edges, compile before running

```
builder = StateGraph(TrialState)
builder.add_node("research", ...)
builder.add_node("evaluate", ...)
builder.set_entry_point("intake")

builder.add_conditional_edges(
    "orchestrator", router, {...})

graph = builder.compile()
```

what this does

nodes are the agents we wrote
conditional edges enable the routing
they make the bounce-back
loop possible
compile() checks the wiring upfront

LOW TEMP (0)

same question, same answer
careful, predictable, repeatable
best choice for a live demo

HIGH TEMP

same question, varied answers
creative, surprising,
less reliable
good for brainstorming,
not safety

think of it as a **creativity dial**.

ACT 3

THE LIVE RUN

Live: one agent corrects another

- | | | |
|----|---------------------|--|
| 01 | Researcher | first pass looks over-confident. |
| 02 | Evaluator | asks: what are the liver values? |
| 03 | Orchestrator | sends it back with that question. |
| 04 | Researcher | second pass is honestly uncertain. |
| 05 | Finalize | stops safely, flags the open question. |

REMOVE THE DOUBT, WATCH IT FAIL

```
one line changed: EVALUATOR_ENABLED = False
```

GATEKEEPER ON

- flags the unresolved liver risk
- asks for the missing values
- refuses to over-claim eligibility

GATEKEEPER OFF

- declares the patient likely eligible
- no caveat and no open question
- confidently wrong on real risk

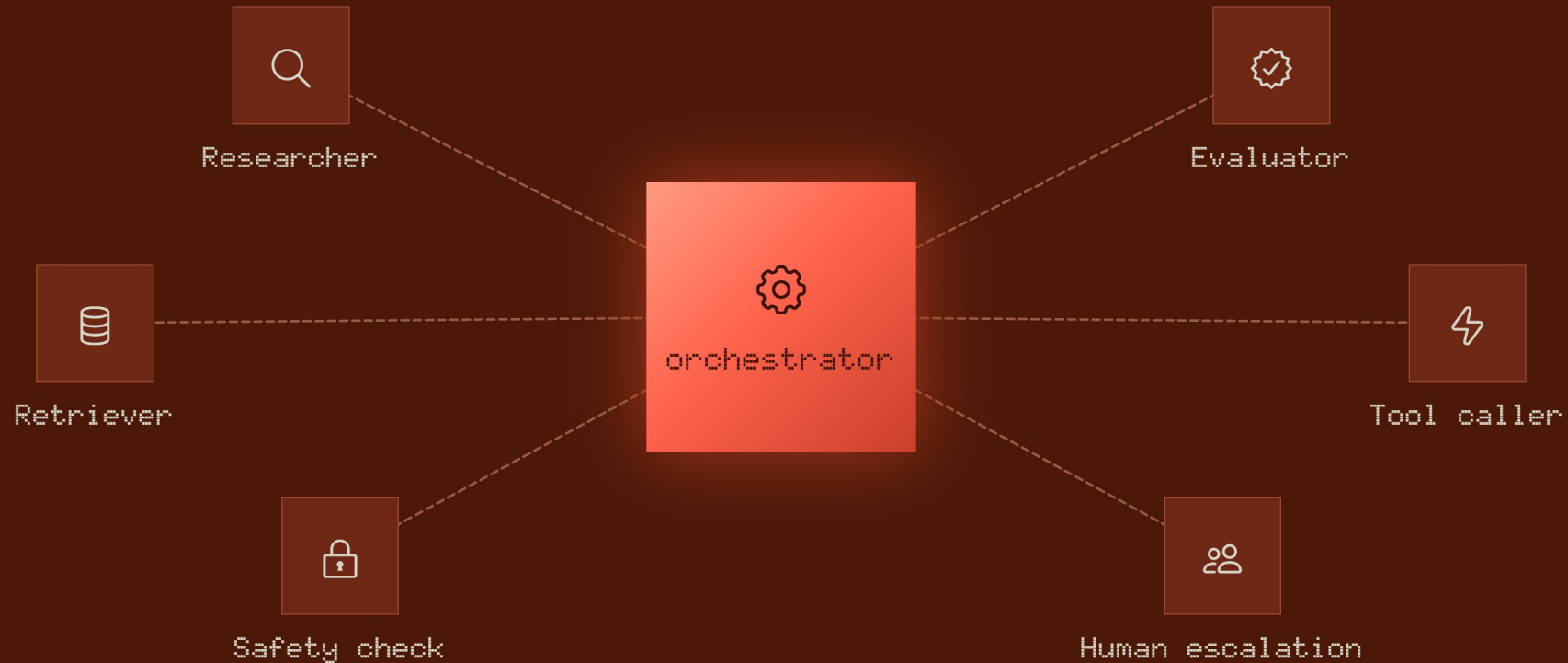
The architecture
is the safety.

ACT 4

BEYOND THE DEMO

Where the orchestrator earns its keep

our demo is small. real systems are not, and the brain gets busy.



It manages agents, tools, retries, budgets, and when to call a human.

Why Groq for this demo

Very fast inference

low latency keeps a live demo snappy

Open models

runs Llama-3.3-70B out of the box

Simple integration

one package, one API key, done

Drop-in via LangChain

ChatGroq plugs straight into the graph

The architecture is model-agnostic. *Swap Groq for any provider.*

WHAT THIS MEANS FOR YOUR WORK

THE RULES

- separate gathering from deciding
- let agents admit being unsure
- make every retry visible
- and capped

WHERE IT FITS

- fintech; loan and KYC diligence
- edtech; student intervention
- triage
- r&d; research paper screening

Open to collaborate on research projects.

The code is yours. Let's build together.

github.com/zacharias1219/Agentic-AI-Demo

References & resources

LangGraph docs

docs.langchain.com/oss/python/langgraph

StateGraph API

reference.langchain.com/python/langgraph

Multi-agent guidance

docs.langchain.com/oss/python/langchain/multi-agent

Groq + LangChain

console.groq.com/docs/langchain

Groq models & limits

console.groq.com/docs/models

Project repo

github.com/zacharias1219/Agentic-AI-Demo